Agent Security and Evaluation in Production

1. The Big Picture: Core Summary

This session from Day 4 of Kaggle and Google's AI Agents Intensive Course focuses on the critical shift required to trust agentic systems in production environments. Unlike traditional software where trust is binary and established once at deployment, autonomous agents require continuous, dynamic security and evaluation. The discussion introduces a comprehensive seven-pillar agent security architecture that treats context as the new perimeter, emphasizing isolated sandboxes, just-in-time downscoped tokens, and defense mechanisms like red/blue/green agent teams. Furthermore, it highlights the necessity of trajectory-aware evaluation—shifting from merely testing final outputs to rigorously tracing the reasoning and steps an agent takes to ensure alignment and prevent "fragile success traps" in vibe coding.

2. ■ Highlights of the Conversation

- **Continuous Trust:** Security in agentic systems cannot be a one-time deployment gate; it must be continually earned and monitored through dynamic context perimeters.
- **Seven-Pillar Architecture:** A robust security framework is necessary to secure supply chains against threats like slop-squatting, ensuring malicious hallucinations don't compromise the system.
- **Network Isolated Sandboxes:** Containing the blast radius of dynamic code is achieved by executing operations inside ephemeral, isolated environments like GVisor.
- **Zero Ambient Authority:** Mitigating the confused deputy problem requires downscoping access tokens just-in-time, granting scripts only the exact data access they need.
- **Agentic Defense Triad:** Security operations can be automated using red team agents for adversarial prompting, blue team agents for monitoring runtime bills of materials, and green team agents for quarantining anomalies and auto-refactoring fixes.
- **Vibe Diff:** For high-stakes operations, compiled syntax is translated back into plain language, allowing humans to confidently sign off on actions.
- **Trajectory-Aware Evaluation:** Evaluating only the final output is dangerous; developers must trace the full sequence of tools and reasoning an agent uses to achieve a result.
- **Agent as a Judge:** Utilizing secondary LLMs or agents to continuously evaluate trajectories, optimize costs, and ensure alignment without pushing evaluation to the very end of the software development lifecycle.

3. ■ Participants' Backgrounds

- **Smitha Collins:** Senior Developer Relations Engineer at Google Cloud. She serves as the primary host for the course, guiding the discussion and introducing the core concepts of the day.
- **Anant Nawalgaria:** Senior Staff Machine Learning Engineer at Google and the founder of the course. He specializes in agentic workflows and production-grade agent deployment, presenting the foundational theories from the whitepapers.
- **Socrates:** A whitepaper co-author and Google expert in security architectures, focusing on integrating security observability directly into software development lifecycles.
- **Meltem Subasioglu:** Google Cloud Generative AI Solutions Architect, researcher, and whitepaper co-author (including "Agent Quality"). Her focus is on rigorous AI evaluation and ensuring the quality of

non-deterministic AI agents.

• **Wafa:** A whitepaper co-author and Google researcher whose expertise includes evaluating agent plans before code generation and implementing tracing tools for agentic applications.

4. ■■ Questions, Answers, and Connections

Question: So, how do you think enterprise leaders can transform AI security and email from a restrictive gate into a competitive business enabler, integrating it into the dev workflows to unlock, as we say at Google, 10x productivity from white coding?

• **Connection to Topic:** This question addresses the fundamental shift from treating security as an afterthought to integrating it directly into the Agentic Engineering workflow, demonstrating how the seven-pillar security architecture operates in practice.

Highlights:

• **Shift in Paradigm:** Moving from seeing security as a one-off final step to integrating it seamlessly into the CI/CD pipeline.

• **Agentic Engineering:** Standardizing workflows so that every code deployment to a sandbox includes automated attacking, observing, and fixing mechanisms.

• **Human in the Loop:** Acknowledging that highly sensitive production environments still require the "Vibe diff" for human approval of clear, readable security reports.

> *Amazing question. This is something that every customer used to ask to us, right? White coding generate massive amount of artifacts code, but most of the times, most of the developers, they treat security and evaluation as the last step of the journey, one-off operation. That is not really the case here. We need to integrate the fixing, the observability of security within the software development life cycle. And specifically, as you mentioned earlier, we have red, blue, green team. The red team is the attacker, blue green is observer, green team is the fixer. So, we need to build this fixer in order to start the automation process. What is the automation process? As we saw in day one, uh we need to move from Vibe coding to Agentic Engineering. Agentic Engineering means that you needs to standardize everything using CSD pipelines. And within the CSD pipelines, every time that you produce code and you deploy that to a sandbox with all the just-in-time tokens as you mentioned, you need also to integrate the process of attacking, [clears throat] observing, and fixing security measures. With that way now, it is not the last step of your journey, the security resolution, but is part of the software development. Of course, you cannot fully automate everything every time. You have still some production environments that are extremely restrictive and extremely sensitive. In that case, you need to use the Vibe diff, but the reports that every human can read and can understand, here's the report of the security, here are the issues, here are what we observe. Do you approve or reject this to go into production?*

Question: Since correct final outputs can hide dangerously flawed reasoning, how can developers implement trajectory aware evaluation to catch hidden side effects of generated code before this before the code is executed in production?

• **Connection to Topic:** This question highlights the difference between traditional testing (which only checks the final output) and the observability required for AI agents to ensure they arrive at the correct answer through secure, intended methods rather than harmful workarounds.

Highlights:

- **Fragile Success Traps:** Agents might hack around a problem (e.g., loading excessive data into memory instead of optimizing a database query) yielding a seemingly correct final output through flawed reasoning.
- **Logging Reasoning Steps:** Developers should instruct the agent to detail its actions at every step, capturing these intermediate reasoning steps via open telemetry.
- **Monitoring Tool Utilization:** It is crucial to log which tools the agent calls, the parameters injected, and how the agent interprets the tool responses to evaluate true alignment.

> *I can go first. Um I think also lines with what I said previously. Um maybe first to give an example so folks can understand what uh such a critical failure mode could be. We also call these fragile [snorts] success traps. So, think about your building with your agent and your app coding. And essentially your code you also have tool codes to to databases, for example. And then you realize, "Hey, the call to the database somehow takes very long and let's optimize it." And you also build your test cases with the agent to test if the call like the latency optimized. You set all of this up. And essentially what then the agent is doing, what is also kind of called clever Hans in research, think of it of the agent then saying, "Okay, instead of like optimizing it properly, what I'm going to do is I'm going to load, I don't know, 100k rows into the local memory. So, whenever you make a call and you cache it's stored and it's faster for you." Which didn't solve the problem. Right? It solved it in a way that when you then test it, the final output seems correct. So, it's also passing the test that you maybe have written. It looks like you have optimized uh your latency with this. But essentially what it did instead was it hacked its way around it. So, it's really important that you not only treat this as a black box. So, don't just evaluate on kind of the final output you're seeing, but evaluate on the trajectory and what we call also a viable trajectory. Which also means that you of course need to have the knowledge to evaluate on it. So, the recommendation is of course to gather as much intel and input as you can. Meaning what I recommend on doing also when I'm white coding myself is on the first level instruct agent to tell you what it's doing at each step of the way. You can then also log these things and and and open telemetry or capture it wherever you're operating the agent from. And essentially also not just logging these intermediate reasoning steps, but also logging what is happening with the agent itself. Like what tools are being called, what parameters are being injected into the into the tool call. You can also look into the tool responses and how the agent interpreted the tool responses. So make sure that you log as much information as possible because this will help help you assess and understand what the agent was doing and if this was aligned with what you wanted it to do. Which as previously mentioned you can do with agent as judges that can support you with your evaluation on different metrics.*